CadmIA Experiments

Tick-Counter Reordering Study under Interval Arithmetic

Damián Vicino

2026

Contents

1 Implementation Notes

This section records the design decisions and invariants of the CadmIA implementation of IA-DEVS that apply to every experiment in this repository.

Foundational References

The experiments in this repository reproduce models from the literature; each experiment section cites its source. UA-DEVS and IA-DEVS are defined in [1], which is the primary reference for all formalism definitions used here.

Interval Representation

IA-DEVS replaces every scalar quantity with an *interval*. CadmIA represents an interval as a pair $[\ell, u]$ where ℓ and u are values of the same underlying scalar type. Any interval kind is permitted:

Kind	Condition
Closed	$\ell \leq u$
Open	$\ell < u$ (endpoints excluded)
Half-open	one endpoint excluded
Point	$\ell = u$ (point interval)
Empty	$\ell > u$ (no element)

There is no requirement that intervals be closed. The choice of interval kind is part of the model specification for each experiment and should be justified there.

TIME Type

The type used for time values in a CadmIA model may be *any type that is a valid TIME type in Cadmium or CDBoost*: floating-point types (`float`, `double`), fixed-point rational types (`decimal<N>`), integer types, or any user-defined totally-ordered arithmetic type.

The interval type for time is therefore `interval<T>` for any such `T` — not exclusively `interval<decimal<N>>`. The choice of `T` is an approximation decision recorded in each experiment's spec.

Outward-Rounding Invariant

The fundamental soundness requirement for IA-DEVS is that every interval must *contain* the exact value it approximates. Formally, if r is an exact real value and $[\ell, u]$ is its representation:

$$\ell \leq r \leq u$$

Equivalently, rounding must always go *outward*: the lower bound rounds toward $-\infty$ (never up), the upper bound rounds toward $+\infty$ (never down). Rounding toward the inside of the interval is not permitted, as it would exclude the true value and invalidate the IA-DEVS enclosure guarantee.

Consequences by TIME type

`decimal<N>`. Values expressible as multiples of 10^{-N} are represented exactly; no rounding occurs and intervals may be point intervals. All experiments in this repository that use `decimal<3>` with periods that are exact multiples of 1 ms satisfy this condition automatically.

Floating-point (`float`, `double`). IEEE 754 arithmetic does not guarantee outward rounding by default. When constructing an interval endpoint from a computed value, the caller must use directed rounding:

- Lower bound: round toward $-\infty$ (`fesetround(FE_DOWNWARD)`)
- Upper bound: round toward $+\infty$ (`fesetround(FE_UPWARD)`)

Experiments that use floating-point TIME must document whether directed rounding is applied, and if not, what approximation error is being accepted.

Integer types. Integers are exact; intervals may always be point intervals when the modelled quantity is an integer.

Portless Coupling

UA-DEVS and IA-DEVS, as defined in VDW21, do not include ports in their formalism. Ports are a feature of PDEVS-with-ports (the formalism Cadmium implements), not of DEVS or its uncertainty extensions. CadmIA therefore connects components via a single untyped channel per pair; the message *value* distinguishes signal semantics.

This is a correct implementation of the formalism, not a missing feature. Each experiment spec must document the value-to-signal encoding it uses.

SELECT Function

When multiple components are simultaneously imminent, the CadmIA coordinator calls a user-supplied SELECT function to pick the next component to advance. The choice affects the simulation trace whenever simultaneous events carry causal dependencies. Each experiment spec must state the SELECT policy and justify it.

2 VDW14 Tick-Counter

Overview

The tick-counter model from [2] simulates a periodic tick source counted over time and periodically reset. A fast generator fires every 1/10 s, a slow generator fires every 1 s, and a counter accumulates ticks and outputs the count on each reset; ideally the count is 10 at every reset. This experiment applies the IA-DEVS formalism [1] to the same model, replacing scalar time with interval arithmetic and studying whether the enclosure guarantee eliminates causality errors.

UA-DEVS Model

The tick-counter is a coupled model $N = \langle D, \{M_d\}, \mathrm{IC}, \mathrm{EOC}\rangle$ with $D = \{G_{1/10},\, G_1,\, K\}$.

Generator G_p (parametric)

$$G_p = \langle S_G,\, T,\, \emptyset,\, Y_G,\, \delta^G_{\mathrm{int}},\, \lambda^G,\, ta^G\rangle$$

$$\begin{aligned} S_G &= \{\bullet\} \quad \text{(single state)} \\ ta^G(\bullet) &= p \in T \quad \text{(period, a TIME value)} \\ \delta^G_{\mathrm{int}}(\bullet) &= \bullet \\ \lambda^G(\bullet) &= \text{tick signal} \end{aligned}$$

$G_{1/10}$ uses $p = 1/10$ and outputs a tick value of 1. G_1 uses $p = 1$ and outputs a reset signal.

Counter K

$$K = \langle S_K,\, T,\, X_K,\, Y_K,\, \delta^K_{\mathrm{int}},\, \delta^K_{\mathrm{ext}},\, \lambda^K,\, ta^K\rangle$$

$$\begin{aligned} S_K &= \mathbb{N} \times \{\text{idle, ready}\} \\ ta^K(n, \text{idle}) &= \infty \\ ta^K(n, \text{ready}) &= 0 \\ \delta^K_{\mathrm{ext}}((n, \text{idle}),\, e,\, x) &= \begin{cases} (n + x,\ \text{idle}) & x \neq 0 \quad \text{(tick)} \\ (n,\ \text{ready}) & x = 0 \quad \text{(reset signal)} \end{cases} \\ \delta^K_{\mathrm{int}}(n, \text{ready}) &= (0,\ \text{idle}) \\ \lambda^K(n, \text{ready}) &= n \end{aligned}$$

Coupling

Both generators route to K. The coupling notation below uses named channels for clarity; the IA-DEVS implementation encodes the signal in the message value (see implementation notes).

$$\begin{aligned} \text{IC} &= \{(G_{1/10}.\texttt{out} \to K.\texttt{add}),\ (G_1.\texttt{out} \to K.\texttt{reset})\} \\ \text{EOC} &= \{(K.\texttt{out} \to N.\texttt{out})\} \end{aligned}$$

Selected Approximations

TIME type. `decimal<3>` (millisecond resolution). All periods in this experiment are exact multiples of 1 ms, so no rounding occurs and the outward-rounding invariant is satisfied trivially.

Interval kind for periods. Point intervals. The periods $p_{G_{1/10}} = 100\,\text{ms}$ and $p_{G_1} = 1000\,\text{ms}$ are assumed known exactly. Choosing point intervals eliminates period uncertainty from the model so that any causality error must arise from time arithmetic rather than from model specification.

Interval kind for values. Point intervals. Tick value 1 and reset signal 0 are exact integers; both are represented as point intervals $[1, 1]$ and $[0, 0]$.

IA-DEVS Resulting Model

With the approximations above, the interval counterparts of the transitions are:

Generator (interval)

$$\begin{aligned} \tilde{ta}^{G}(\tilde{\bullet}) &= [p,\ p] \quad \text{(point interval period)} \\ \tilde{\delta}^{G}_{\text{int}}(\tilde{\bullet}) &= \tilde{\bullet} \\ \tilde{\lambda}^{G}(\tilde{\bullet}) &= [v,\ v] \quad (v = 1 \text{ for } G_{1/10},\ v = 0 \text{ for } G_1) \end{aligned}$$

Counter K (interval)

Let $\tilde{n} \in \mathcal{I}(\mathbb{N})$.

$$
\begin{aligned}
\tilde{ta}^K(\tilde{n}, \text{idle}) &= [\infty, \infty] \\
\tilde{ta}^K(\tilde{n}, \text{ready}) &= [0, 0] \\
\tilde{\delta}^K_{\text{ext}}((\tilde{n}, \text{idle}),\, \tilde{e},\, \tilde{x}) &= \begin{cases} (\tilde{n} + \tilde{x},\ \text{idle}) & 0 \notin \tilde{x} \\ (\tilde{n},\ \text{ready}) & \tilde{x} = [0, 0] \end{cases} \\
\tilde{\delta}^K_{\text{int}}((\tilde{n}, \text{ready})) &= ([0, 0],\ \text{idle}) \\
\tilde{\lambda}^K(\tilde{n}, \text{ready}) &= \tilde{n}
\end{aligned}
$$

Expected output

Because all intervals are point intervals and arithmetic is exact under `decimal<3>`, every internal time is a point:

$$\tilde{t}_k^{G_{1/10}} = [k/10,\, k/10], \qquad \tilde{t}_k^{G_1} = [k,\, k]$$

At each integer second both generators are simultaneously scheduled; after ten tick events the counter holds:

$$\tilde{n} = 10 \cdot [1, 1] = [10, 10]$$

K always outputs the point interval [10, 10].

Implementation Notes (CadmIA)

Portless coupling. UA-DEVS and IA-DEVS do not define ports; coupling is by value. Both generators route to K's single `input_t = int`. The reset signal is encoded as value 0; any non-zero value is a tick.

Interval value $\tilde{x}$	Semantics for K
$[0, 0]$	reset: output count, go to ready
$\tilde{x},\ 0 \notin \tilde{x}$	tick: add $\tilde{x}$ to accumulator

SELECT policy. At $t = [k, k]$ for integer k, both $G_{1/10}$ and G_1 are imminent. $G_{1/10}$ must be selected first so that K accumulates the k-th tick before receiving the reset signal. The SELECT function is:

$$\text{SELECT}(S) = \begin{cases} G_{1/10} & \text{if } G_{1/10} \in S \\ G_1 & \text{if } G_1 \in S \\ \text{first}(S) & \text{otherwise} \end{cases}$$

Implementation status. Implemented and verified. Across 100 resets, K outputs $[10, 10]$ at every reset event (0 errors). Source: `vdw14/main.cpp`.

References

[1] Damián Vicino, Gabriel A. Wainer, Olivier Dalle. *Uncertainty on Discrete-Event System Simulation.* ACM Transactions on Modeling and Computer Simulation (TOMACS), Vol. 32, No. 1, Article 2, September 2021. DOI: 10.1145/3466169.

[2] Damián Vicino, Olivier Dalle, Gabriel A. Wainer. *A Data Type for Discretized Time Representation in DEVS.* In *Proc. 7th Intl. ICST Conf. on Simulation Tools and Techniques (SIMUTOOLS)*, 2014. hal-01055555.